

UNIVERSITY OF THE PUNJAB

Faculty of Computing & Information Technology (FCIT)



Computer Networks (CN)

PROJECT REPORT

NetWatch — Network Traffic Monitoring and Analysis Platform

Submitted By:	Hafiz Muhammad Talal
Roll Number:	BCSF24M002
Program:	BSCS
Session:	2024
Submitted To:	Dr. Madeeha Amaan
Date:	4th May, 2026

1. Introduction

1.1 Project Title

NetWatch..... A Network Traffic Monitoring and Analysis Platform

1.2 Project Description

NetWatch is a real-time network traffic monitoring platform developed as part of the Computer Networks course. The system captures live network packets directly from the active network interface, processes them, and displays the results through a clean web-based interface accessible via any browser.

The platform demonstrates core networking concepts including packet structure, protocol identification, port-to-service mapping, and traffic analysis. It provides both live monitoring capabilities and statistical summaries of captured traffic.

1.3 Objectives

- Capture and display real-time network packets using raw socket sniffing
- Identify and classify packets by protocol type: TCP, UDP, and ICMP
- Map destination port numbers to human-readable service names (e.g., Port 80 → HTTP)
- Provide filtering capabilities by protocol, source IP, and destination IP
- Display live statistics including total packets, per-protocol counts, and average packet size
- Maintain a system log of monitoring events

1.4 Technologies Used

Component	Technology	Purpose
Backend	Python + Flask	Web server and REST API endpoints
Packet Capture	Scapy	Real-time raw packet sniffing
Frontend	HTML / CSS / JS	Web interface — no frameworks used
Threading	Python threading	Background sniffer thread
CORS	Flask-CORS	Cross-origin API access during dev

2. Methodology

This section describes the step-by-step process followed to design and implement the NetWatch platform.

Step 1 — Technology Selection

Python was selected as the primary language due to Scapy's powerful and well-documented packet capture capabilities. Flask was chosen as the web framework because of its simplicity and lightweight nature, making it ideal for serving a small web interface and a set of REST API endpoints. Plain HTML, CSS, and JavaScript were used for the frontend as per the project requirements, without any heavy frameworks.

Step 2 — System Architecture Design

The system was designed with a clear separation between the backend and frontend. The Flask server handles all packet processing and exposes four API endpoints:

- `/api/start` — begins the packet capture session
- `/api/stop` — stops the capture session
- `/api/packets` — returns the filtered packet log as JSON
- `/api/stats` — returns live statistics as JSON

The frontend communicates with these endpoints using the browser's `fetch()` API, polling every one second to refresh the displayed data.

Step 3 — Packet Capture Implementation

Scapy's `sniff()` function was used to capture packets from the active network interface. Since `sniff()` is a blocking call, it was placed inside a background thread so that the Flask web server could continue responding to API requests simultaneously. A `threading.Lock()` object was used to protect the shared packet list and statistics dictionary from race conditions between the sniffer thread and the Flask request handler threads.

Each captured packet is processed as follows:

- The IP layer is extracted to obtain source IP, destination IP, and packet size
- The presence of TCP, UDP, or ICMP layers is checked to determine the protocol
- Source and destination port numbers are extracted for TCP and UDP packets
- The destination port is mapped to a service name using a static dictionary containing 30+ common port mappings
- The processed record is appended to the in-memory packet log

Step 4 — Port-to-Service Mapping

A Python dictionary was implemented to map common port numbers to their corresponding service names. This allows the interface to display meaningful labels such as HTTP, DNS, HTTPS, SSH, FTP, and RDP instead of raw port numbers. The mapping covers 30+ well-known ports as defined in standard networking references.

Step 5 — Frontend Development

The web interface was built using HTML, CSS, and vanilla JavaScript. The interface includes all required components: Start and Stop buttons, a live packet table, a statistics panel, a filter section, and a system log panel. The frontend polls the backend every 1000 milliseconds using `setInterval()` and updates the DOM with the latest packet data and statistics without requiring a full page reload.

Step 6 — Testing

The system was tested by running the Flask application with administrator privileges and observing live network traffic. The following tests were performed:

- General browsing traffic: confirmed TCP and UDP packets captured for HTTPS connections
- DNS queries: confirmed UDP packets to port 53 (8.8.8.8) labeled as DNS
- ICMP traffic: executed ping 8.8.8.8 in Command Prompt and confirmed ICMP packets appeared in the table
- Filter testing: applied TCP, UDP, and ICMP filters and verified correct record counts
- IP filter testing: entered a source IP substring and confirmed only matching packets were displayed

Screenshots:

All components working:

1. All protocols
2. TCP filter
3. UDP filter
4. ICMP filter
5. DNS or IP address filter

The screenshot displays the NetWatch web interface. At the top, there are 'START' and 'STOP' buttons. Below them, statistics are shown: Total Packets: 548, TCP: 184, UDP: 341, ICMP: 23, and Avg Packet Size: 372.5 B. A filter section allows selecting a protocol (All Protocols) and entering source and destination IP addresses (e.g., 192.168.1.5 and 8.8.8.8). The 'LIVE PACKET CAPTURE' table shows 548 records with columns for #, TIME, SOURCE IP, SRC PORT, DESTINATION IP, DST PORT, PROTOCOL, SERVICE, and SIZE (B). The system log at the bottom shows messages like 'Filter applied - Proto: ALL, Src: *, Dst: *' and 'Monitoring stopped'.

#	TIME	SOURCE IP	SRC PORT	DESTINATION IP	DST PORT	PROTOCOL	SERVICE	SIZE (B)
538	04-32-23	192.168.1.4	51772	158.171.109.163	443	TCP	HTTPS	54
537	04-32-23	158.171.109.163	443	192.168.1.4	51772	TCP	Port-51772	54
536	04-32-23	158.171.109.163	443	192.168.1.4	51772	TCP	Port-51772	85
535	04-32-23	158.171.109.163	443	192.168.1.4	51772	TCP	Port-51772	100
534	04-32-23	104.18.41.41	443	192.168.1.4	64060	UDP	Port-64060	70
533	04-32-23	192.168.1.4	64060	104.18.41.41	443	UDP	HTTPS	83
532	04-32-23	1.1.1.1	-	192.168.1.4	-	ICMP	Unknown	70
531	04-32-23	192.168.1.4	62600	104.18.30.10	443	UDP	HTTPS	86
530	04-32-23	192.168.1.4	62600	104.18.30.10	443	UDP	HTTPS	86
529	04-32-23	104.18.30.10	443	192.168.1.4	62600	UDP	Port-62600	576

NETWATCH ● IDLE

▶ START

■ STOP

PACKETS: 548

TOTAL PACKETS
548

TCP
184

UDP
341

ICMP
23

Avg. PACKET SIZE
372.5 B

PROTOCOL
TCP

SOURCE IP
e.g. 192.168.1.5

DESTINATION IP
e.g. 0.0.0.0

FILTER

RESET

LIVE PACKET CAPTURE

200 records

#	TIME	SOURCE IP	SRC PORT	DESTINATION IP	DST PORT	PROTOCOL	SERVICE	SIZE (B)
436	04-32-16	192.168.1.4	50161	150.171.27.11	443	TCP	HTTPS	66
435	04-32-16	192.168.1.4	50161	150.171.27.11	443	TCP	HTTPS	66
434	04-32-16	150.171.27.11	443	192.168.1.4	50161	TCP	Port-50161	255
433	04-32-16	150.171.27.11	443	192.168.1.4	50161	TCP	Port-50161	92
432	04-32-16	150.171.27.11	443	192.168.1.4	50161	TCP	Port-50161	503
431	04-32-16	150.171.27.11	443	192.168.1.4	50161	TCP	Port-50161	1474
430	04-32-16	150.171.27.11	443	192.168.1.4	50161	TCP	Port-50161	1474
409	04-32-16	192.168.1.4	50161	150.171.27.11	443	TCP	HTTPS	54
408	04-32-16	150.171.27.11	443	192.168.1.4	50161	TCP	Port-50161	54
407	04-32-16	150.171.27.11	443	192.168.1.4	50161	TCP	Port-50161	92

SYSTEM LOG

CLEAR

```
[04-32-03] Filter applied - Proto: TCP, Src: **, Dst: **  
[04-32-17] Filter applied - Proto: All, Src: **, Dst: **  
[04-32-27] Monitoring stopped.  
[04-32-05] Packet sniffing started.  
[04-30-18] Monitoring stopped.  
[04-30-15] Packet sniffing started.  
[04-32-15] Filter applied - Proto: ICMP, Src: **, Dst: **
```

NETWATCH ● IDLE

▶ START

■ STOP

PACKETS: 548

TOTAL PACKETS
548

TCP
184

UDP
341

ICMP
23

Avg. PACKET SIZE
372.5 B

PROTOCOL
UDP

SOURCE IP
e.g. 192.168.1.5

DESTINATION IP
e.g. 0.0.0.0

FILTER

RESET

LIVE PACKET CAPTURE

200 records

#	TIME	SOURCE IP	SRC PORT	DESTINATION IP	DST PORT	PROTOCOL	SERVICE	SIZE (B)
520	04-32-23	104.10.30.10	443	192.168.1.4	62600	UDP	Port-62600	804
519	04-32-23	104.10.41.41	443	192.168.1.4	64060	UDP	Port-64060	70
518	04-32-23	192.168.1.4	64060	104.10.41.41	443	UDP	HTTPS	83
517	04-32-22	104.10.41.41	443	192.168.1.4	64060	UDP	Port-64060	70
516	04-32-22	192.168.1.4	64060	104.10.41.41	443	UDP	HTTPS	83
515	04-32-22	104.10.30.10	443	192.168.1.4	62600	UDP	Port-62600	74
514	04-32-22	104.10.30.10	443	192.168.1.4	62600	UDP	Port-62600	73
513	04-32-22	104.10.30.10	443	192.168.1.4	62600	UDP	Port-62600	74
512	04-32-22	104.10.30.10	443	192.168.1.4	62600	UDP	Port-62600	73
511	04-32-22	104.10.30.10	443	192.168.1.4	62600	UDP	Port-62600	74

SYSTEM LOG

CLEAR

```
[04-34-25] Filter applied - Proto: UDP, Src: **, Dst: **  
[04-34-09] Filter applied - Proto: TCP, Src: **, Dst: **  
[04-32-47] Filter applied - Proto: All, Src: **, Dst: **  
[04-32-27] Monitoring stopped.  
[04-32-05] Packet sniffing started.  
[04-30-18] Monitoring stopped.  
[04-30-15] Packet sniffing started.  
[04-32-15] Filter applied - Proto: ICMP, Src: **, Dst: **
```

NETWATCH

● IDLE

▶ START

■ STOP

PACKETS: 548

TOTAL PACKETS

548

TCP

184

UDP

341

ICMP

23

AVG PACKET SIZE

372.5 B

PROTOCOL

▼ ICMP

SOURCE IP

■ e.g. 192.168.1.5

DESTINATION IP

■ e.g. 0.0.0.0

FILTER

RESET

LIVE PACKET CAPTURE

21 PROVISION

#	TIME	SOURCE IP	SRC PORT	DESTINATION IP	DST PORT	PROTOCOL	SERVICE	SIZE (B)
445	04:32:16	142.250.202.174	—	192.168.1.4	—	ICMP	Unknown	78
444	04:32:16	192.168.1.4	—	142.250.202.174	—	ICMP	Unknown	74
442	04:32:16	0.0.0.0	—	192.168.1.4	—	ICMP	Unknown	74
439	04:32:16	192.168.1.4	—	0.0.0.0	—	ICMP	Unknown	74
379	04:32:15	0.0.0.0	—	192.168.1.4	—	ICMP	Unknown	74
374	04:32:15	192.168.1.4	—	0.0.0.0	—	ICMP	Unknown	74
338	04:32:14	0.0.0.0	—	192.168.1.4	—	ICMP	Unknown	74
324	04:32:14	192.168.1.4	—	0.0.0.0	—	ICMP	Unknown	74
297	04:32:13	0.0.0.0	—	192.168.1.4	—	ICMP	Unknown	74
296	04:32:13	192.168.1.4	—	0.0.0.0	—	ICMP	Unknown	74

SYSTEM LOG

CLEAR

[04:34:58] Filter applied - Proto: ICMP, Src: "", Dst: ""
[04:34:29] Filter applied - Proto: UDP, Src: "", Dst: ""
[04:34:40] Filter applied - Proto: TCP, Src: "", Dst: ""
[04:32:47] Filter applied - Proto: ALL, Src: "", Dst: ""
[04:32:27] Monitoring stopped.
[04:32:05] Packet sniffing started.
[04:30:18] Monitoring stopped.
[04:30:15] Packet sniffing started.

NETWATCH

● IDLE

▶ START

■ STOP

PACKETS: 548

TOTAL PACKETS

548

TCP

184

UDP

341

ICMP

23

AVG PACKET SIZE

372.5 B

PROTOCOL

▼ UDP

SOURCE IP

■ e.g. 192.168.1.5

DESTINATION IP

■ 0.0.0.0

FILTER

RESET

LIVE PACKET CAPTURE

6 PROVISION

SYSTEM LOG

CLEAR

[04:35:38] Filter applied - Proto: UDP, Src: "", Dst: "0.0.0.0"
[04:34:58] Filter applied - Proto: ICMP, Src: "", Dst: ""
[04:34:29] Filter applied - Proto: UDP, Src: "", Dst: ""
[04:34:40] Filter applied - Proto: TCP, Src: "", Dst: ""
[04:32:47] Filter applied - Proto: ALL, Src: "", Dst: ""
[04:32:27] Monitoring stopped.
[04:32:05] Packet sniffing started.
[04:30:18] Monitoring stopped.

3. Challenges Faced

The following challenges were encountered during the development of NetWatch and the corresponding solutions applied:

#	Challenge	Problem	Solution
1	Thread Safety	Scapy's sniffer runs on a separate thread and writes to the same packet list that Flask reads from, creating a potential race condition.	Wrapped all read and write operations on shared state with <code>threading.Lock()</code> to ensure only one thread accesses the data at a time.
2	Blocking Sniffer	Scapy's <code>sniff()</code> function blocks indefinitely by default, making it impossible to stop the capture cleanly when the Stop button is pressed.	Used the <code>timeout=2</code> parameter inside a while monitoring: loop. The thread checks the stop flag every 2 seconds and exits gracefully.
3	Root Privileges Required	Raw packet capture requires administrator or root privileges. Running the app without them causes a <code>PermissionError</code> and the sniffer fails silently.	Documented the requirement clearly in the README. Added <code>try/except</code> blocks around packet processing to handle permission errors without crashing the server.
4	Memory Growth	Continuous packet capture accumulates thousands of records in memory, slowing down the browser table rendering over time.	Capped the in-memory packet log at 500 packets using a list slice. The frontend table renders only the latest 200 entries for performance.
5	CORS Issues	During development and testing, the browser blocked API calls from the frontend to the Flask backend due to Cross-Origin Resource Sharing restrictions.	Installed and configured the <code>flask-cors</code> library to allow cross-origin requests from the frontend to the backend API.

4. Future Work

If more time and resources were available, the following improvements and extensions would be made to the NetWatch platform:

4.1 Persistent Storage

Currently, all captured packets are stored in memory and are lost when the server is stopped. A future version would integrate a SQLite database or write packets to a CSV file during capture, allowing users to review historical sessions and export data for offline analysis.

4.2 Real-Time Charts and Visualizations

Adding live-updating charts using Chart.js or a similar library would greatly improve the usefulness of the statistics panel. A protocol distribution pie chart, a packets-per-second line graph, and a top source IPs bar chart would provide much richer insight into network behavior.

4.3 GeoIP Lookup

Integrating a GeoIP database (such as MaxMind GeoLite2) would allow the platform to resolve destination IP addresses to their geographic location, including country, city, and ISP. This would make it much easier to identify unusual outbound connections.

4.4 Alert and Anomaly Detection System

A rule-based alert system could be added to flag suspicious traffic patterns, such as a high rate of connection attempts to a single port (potential port scanning), repeated ICMP packets (ping flooding), or connections to known malicious IP ranges.

4.5 PCAP Export

Allowing users to download captured traffic as a .pcap file would enable integration with professional tools like Wireshark for deeper packet inspection and protocol analysis beyond what the web interface provides.

4.6 Network Interface Selection

The current implementation sniffs on the default network interface. A future version would allow users to select from all available network interfaces through the web interface, making the tool useful on machines with multiple network adapters.

4.7 BPF Filter Input

Adding a text field to accept Berkeley Packet Filter (BPF) expressions would give advanced users fine-grained control over which packets are captured, without having to modify the source code.

5. References

5.1 Libraries and Frameworks

Library	Version	License	Purpose
Flask	3.0.3	BSD	Web server and REST API
Scapy	2.5.0	GPL-2.0	Real-time packet capture and parsing
Flask-CORS	4.0.1	MIT	Cross-Origin Resource Sharing support
Python threading	stdlib	PSF	Background sniffer thread management
Python collections	stdlib	PSF	defaultdict for statistics

5.2 Course Reference

- Kurose, J. F., & Ross, K. W. — Computer Networking: A Top-Down Approach (7th Edition)
- Chapters 1 & 2: Application Layer, Transport Layer, and Network Layer concepts applied throughout this project

5.3 Online Resources

- Scapy Official Documentation — <https://scapy.readthedocs.io>
- Flask Official Documentation — <https://flask.palletsprojects.com>
- IANA Port Number Registry — <https://www.iana.org/assignments/service-names-port-numbers>
- Npcap Windows Packet Capture — <https://npcap.com>